



Computer Graphics

Dingzhuanlian
Anhui University

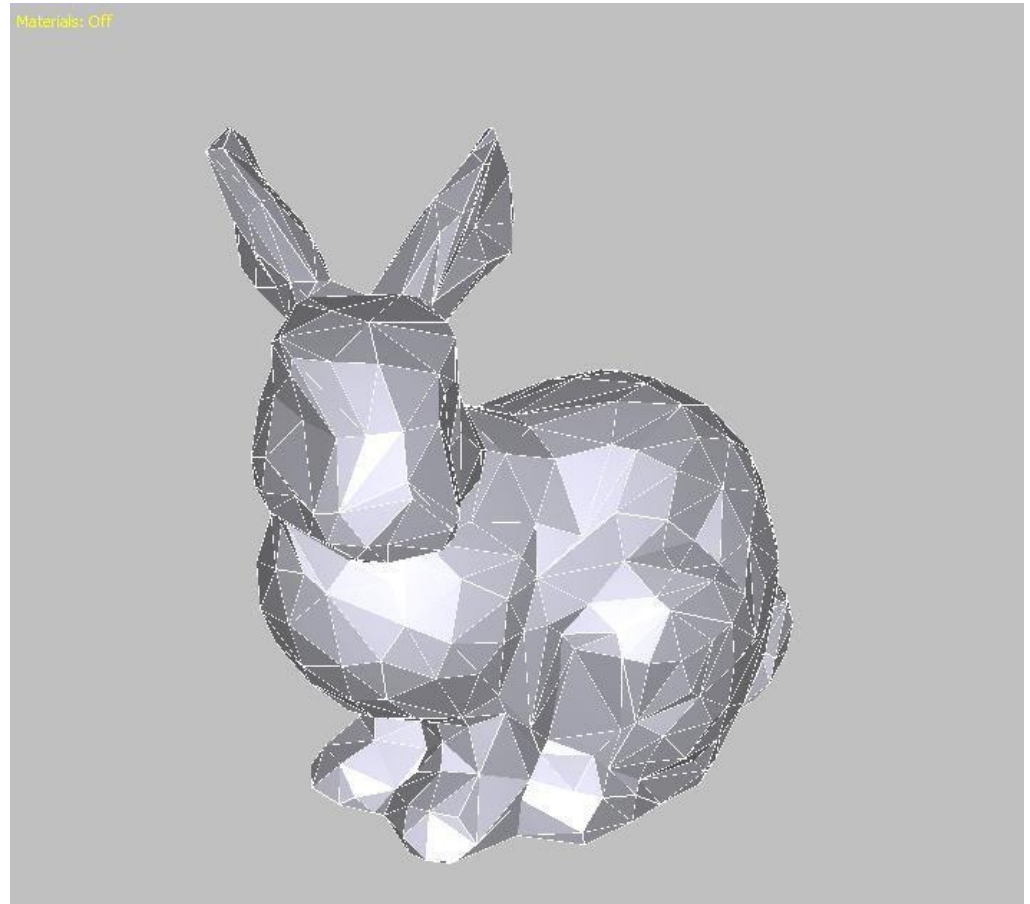


Lighting model



Rendering results

- Wireframe
(线框效果)
- Constant Color
(颜色效果)
- Lighting
(光照效果)





Lighting model

- 当物体的几何形态确定之后，光照决定了整个场景的显示结果。因此，**真实感图形的生成**取决于如何建立一个合适的**光照明模型**（illumination model）。
- 光照明模型：模拟物体表面的光照明物理现象的数学模型。
- 光照明模型的提出使得 从光栅图形学跳到真实感图形学。



Lighting model

- To calculate the intensity of light, we need an lighting model (illumination model)
(光照明模型用于计算光的强度)
 - Local Lighting(局部光照明)
 - Concerned with how objects are directly illuminated by light sources(关注物体直接受到光源影响所产生的光照效果)。
 - Global Lighting(全局光照明)
 - Includes shadow effects(关注阴影效果)
 - Includes lighting effects from locations other than light sources, such as reflections, refractions(关注所有不是直接与光源位置相关的光照效果，例如反射和折射效果)



History of Lighting model

- In 1967, Wylie: first added lighting effects into rendering (第一次在显示物体时加入了光照明效果)
 - Intensity is inverse of the distance to the light(光的强度与物体到光源的距离成反比)
- In 1970, Bouknight: introduced the first lighting model(第一个光反射模型):
 - Lambert diffuse reflection(漫发射) + ambient light(环境光) (No specular lighting)

Communication of ACM



History of Lighting

- In 1971, Gourand: gourand shading(漫反射模型加插值的思想)
 - Lambert diffuse + Bicentric interpolation(双心插值)
 - IEEE transactions on Computers
- In 1975, Phong: proposed extended the model by further considering specular (**Phong 模型**)
 - The most influenced lighting model
 - Diffuse light (漫发射) + ambient light (环境光) + specular light (高光)
 - 增加了高光项, 当光线照到光滑表面, 如照射到镜子上, 就会看到一个光斑, 就是高光。开创了真实感图形学的历史, 可以模拟很逼真的图形效果了。
 - Communication of ACM



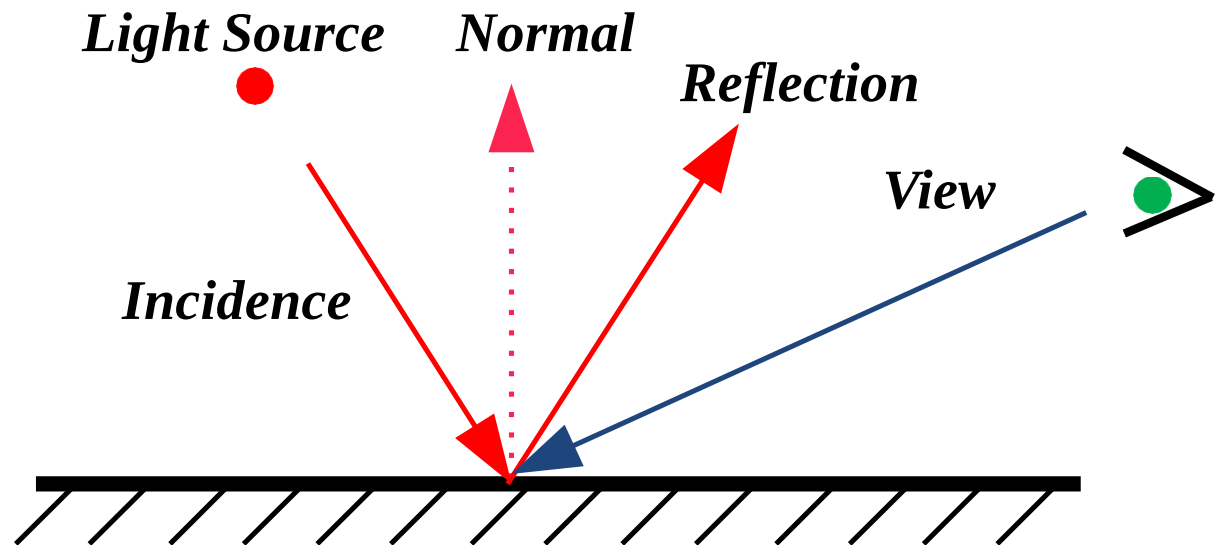
Related Physics(背景物理知识)

- ❑ The propagation of light(光的传播规律, 反射)
- ❑ The law of refraction (折射定律)
- ❑ Diffuse reflection(漫反射)
- ❑ Specular reflection(镜面反射)
- ❑ Ambient Light (环境光)
- ❑ Energy(光线传播的能量方程)



The propagation of light(光的传播规律)

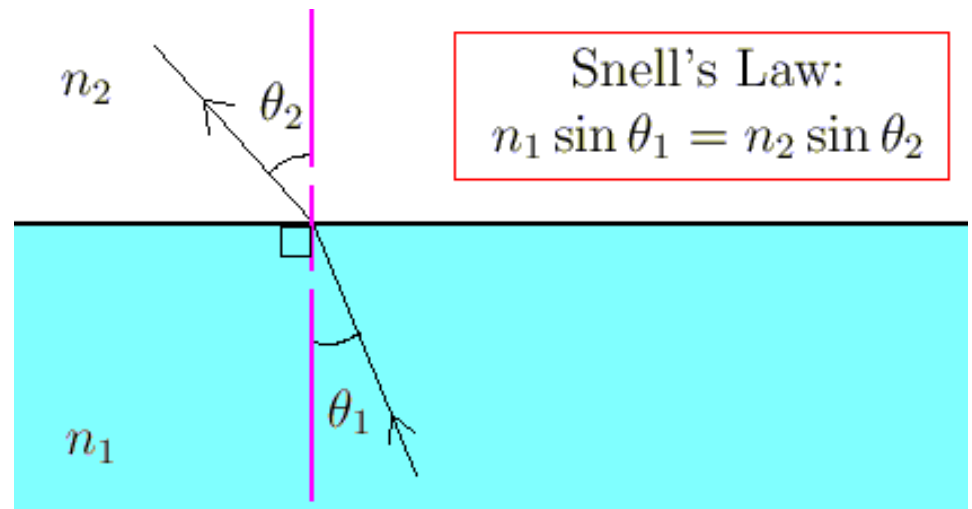
- The propagation of lights obeys law of reflection(反射定律):
 - the angle of incidence = the angle of reflection
(入射角等于反射角)
 - incoming ray, reflection ray and normal is in the same plane(入射光线、反射光线、以及反射面的法向量位于同一平面内)





The law of refraction (折射定律)

- The law of refraction (折射定律, 也叫 Snell 定律):
 - 光从一个介质进入到另一个介质的时候, 会发生偏转.
 - the ratio of the sin of the angles of incidence and of refraction is a constant that depends on the media (入射角和折射角的正弦值之比是一个仅仅取决于介质的常数)
 - The constant is called the relative refractive index(相对折射系数)





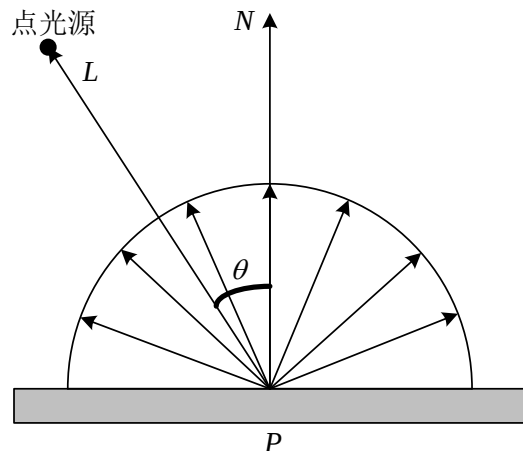
Diffuse reflection(漫反射)

- 物体表面同时存在漫反射和镜面反射。漫反射：光线照到表面，向四面八方散开。我们能看见物体本身的颜色、轮廓，全靠漫反射，亮度均匀、柔和。
- 指光线照射到粗糙表面时，光线被反射后向各个方向散射的现象。
- 特点：光源来自一个方向，反射光均匀地射向各个方向，与视点无关。
- 由Lambert余弦定理可得点P处漫反射光的强度为：

$$I_d = I_p K_d \cos \theta, \theta \in [0, \frac{\pi}{2}]$$

- 若L和N都已规格化为单位矢量，则有

$$I_d = I_p K_d (L \cdot N)$$



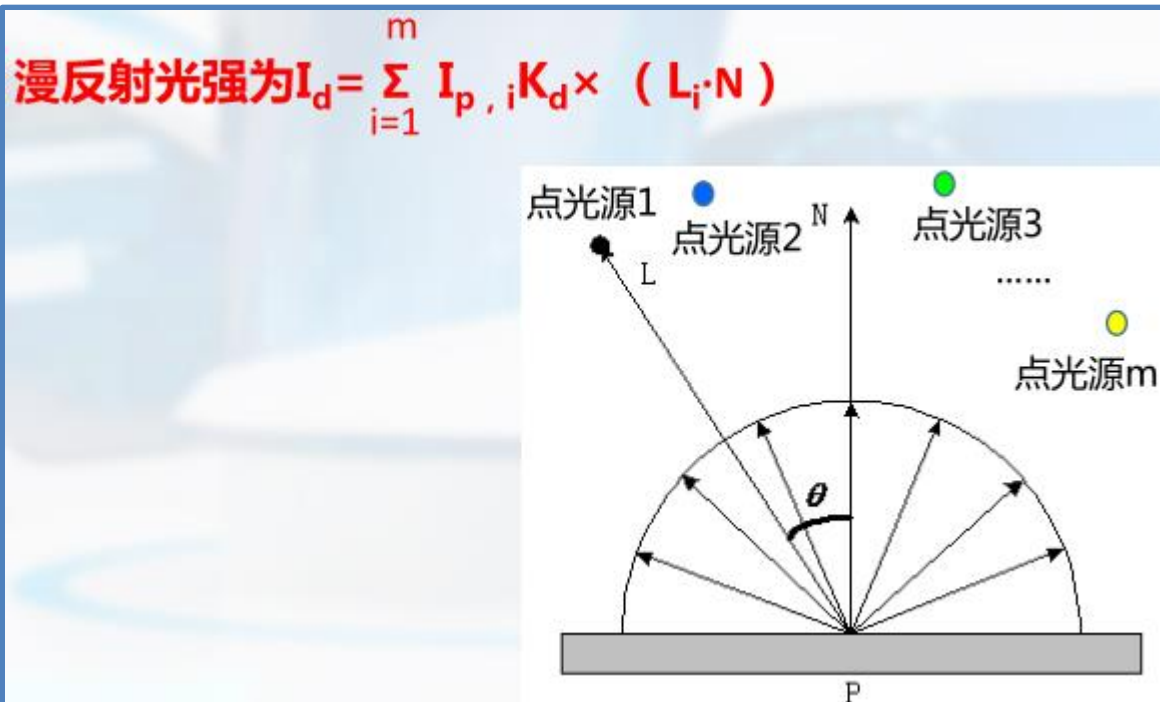
图：漫反射

I_p 是入射光的强度， K_d 是与物体有关的漫反射系数，漫反射系数反映了物体的颜色。漫反射光的颜色由入射光的颜色和物体表面颜色共同决定。



Diffuse reflection(漫反射)

多个点光源 (m 个点光源)





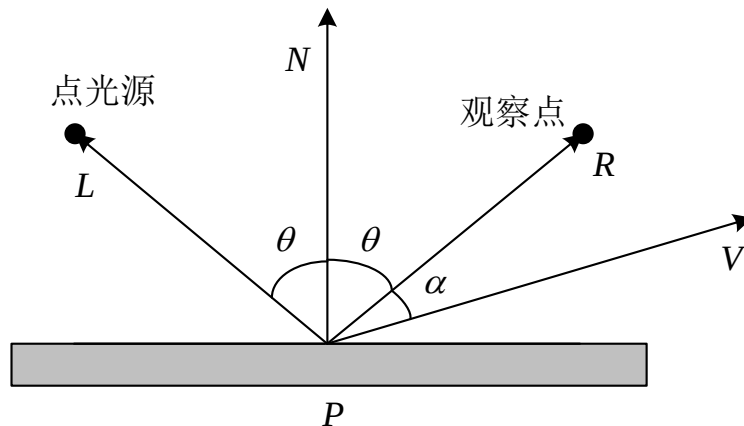
Specular reflection(镜面反射)

- 镜面反射遵循反射定律，入射光和反射光分别位于表面法矢的两侧。
- 对于一般光滑表面，反射光集中在一个范围内，方向集中（聚集）。
- 如果观察者正好处在P点的镜面反射方向上，就会看到一个比周围亮得多的**高光点**。镜面反射光会在反射方向附近形成很亮的光斑，即**高光现象**。
- 镜面反射情况由Phong模型给出：

$$I_s = I_p K_s \cos^n \alpha$$

- 若R和V已规格化为单位矢量，则：

$$I_s = I_p K_s (R \cdot V)^n$$



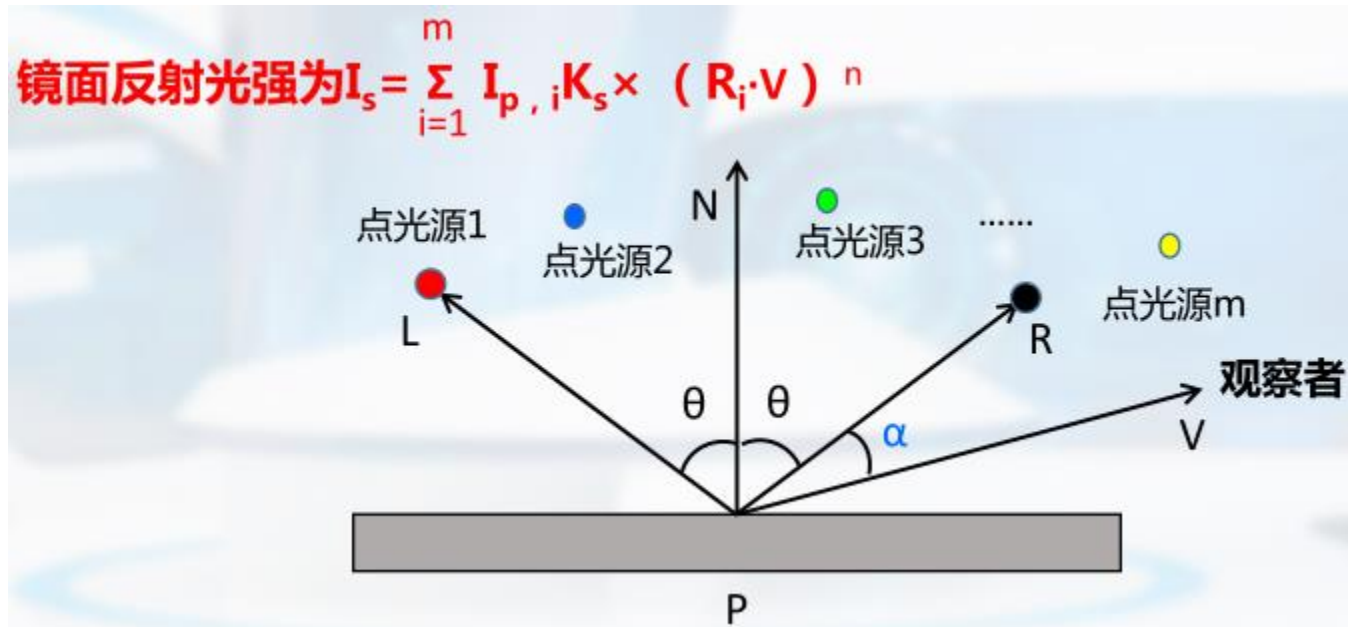
图：镜面反射

I_p 是入射光的强度， K_s 是**与物体有关的镜面反射系数**， n 为**反射指数**，**反映物体表面的光滑程度**，数目越大物体表面越光滑。镜面反射光产生的**高光区域只反应光源的颜色**。镜面反射系数是一个与**物体颜色无关的参数**。



Specular reflection(镜面反射)

多个点光源 (m 个点光源)

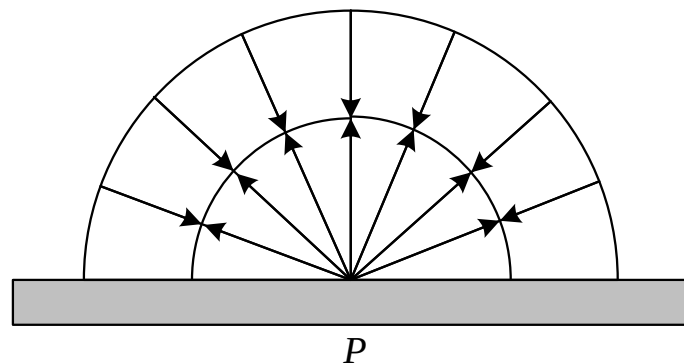




Ambient Light (环境光)

- 在没有光源的地方，景物没有受到光源的直接照射，但其表面仍具有一定的亮度，使它们可见。这是因为光线在场景中经过复杂的传播之后，形成了弥漫于整个空间的光线，称为**环境光**。
- 特点：照射在物体上的光来自周围各个方向，又均匀地向各个方向反射。
- P 点对**环境光的反射强度**为

$$I_e = I_a K_a$$



图：环境光的反射

I_a 环境光强度， K_a 为物体对环境光的反射系数



Energy(光线传播的能量方程)

- Propagation of lights should satisfy Energy conservation(遵循能量守恒定律):

$$I_i = I_d + I_s + I_t + I_v$$

- I_i is the energy of incident light (入射光的能量)
- I_d is the energy of diffuse reflection (漫反射光的能量)
- I_s is the energy of specular reflection (镜面反射光的能量)
- I_t is the energy of refraction (折射光的能量)
- I_v is the energy that is absorbed (被介质和物体所吸收的能量)

□ 吸收光：比如冬天晒太阳会感觉到温暖，这是因为吸收了太阳光。



Lighting model

- **Phong Lighting Model (Phone光照模型)**
- **Phong Shading(增量式光照模型, Gouraud明暗处理)**
- **Gouraud Shading(增量式光照模型, Phong明暗处理)**
- **整体光照模型 (Ray Tracing, 光线跟踪算法)**



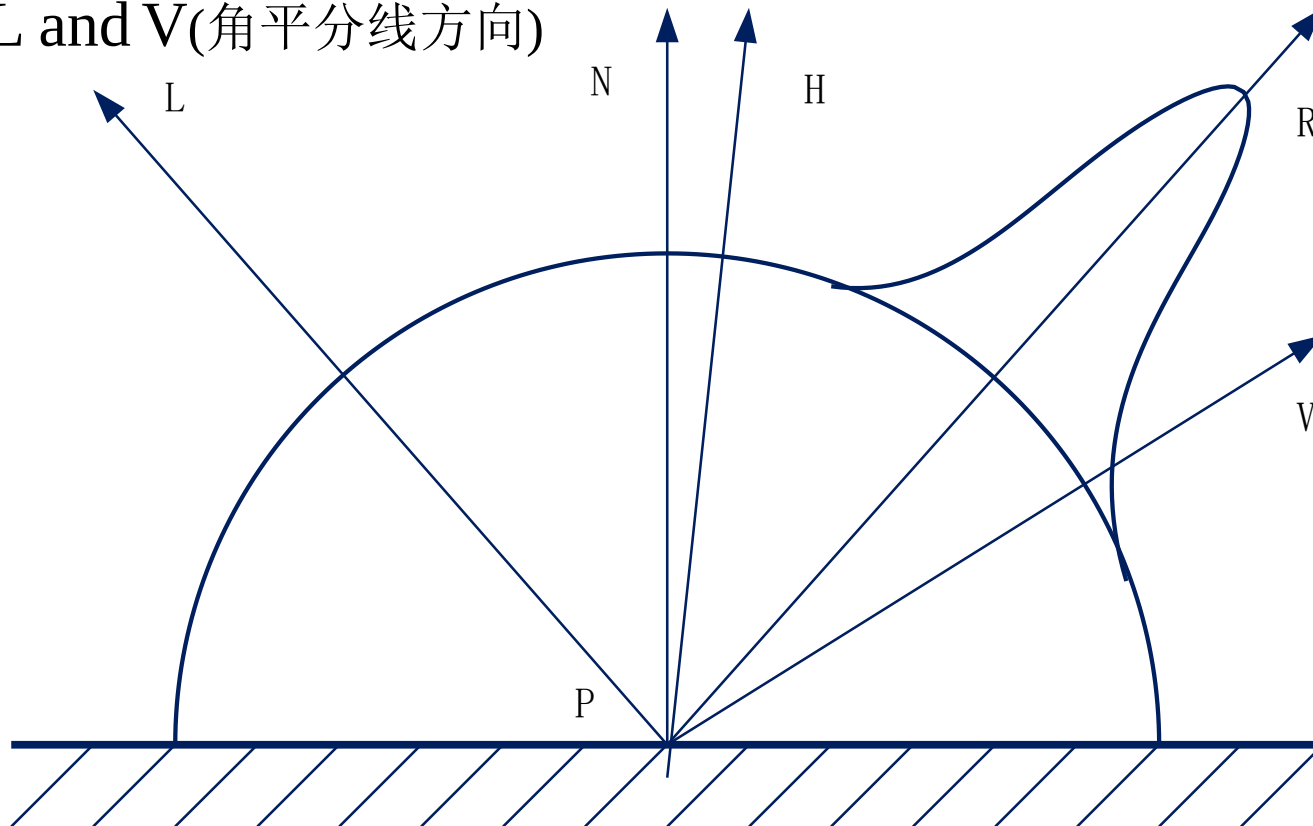
Phong Lighting Model

- Local lighting model (简单的光照模型模拟物体表面对光的反射作用, 它只考虑物体对直接光照的反射作用, 而物体间的反射光只用环境光统一表示)
- Support point light (点光源)
 - Decompose lighting effects to 3 parts:
 - Diffuse Reflection(漫反射光效果)
 - Specular Reflection(镜面反射光效果)
 - Ambient lighting(环境光效果)

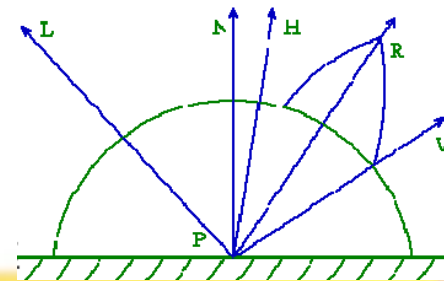


Phong Lighting Model Illustration

L is the incidence ray(入射光), V means the direction of the viewer, N is the normal, and R is reflection of L; **H** is the half of L and V(角平分线方向)



Phong Lighting Model

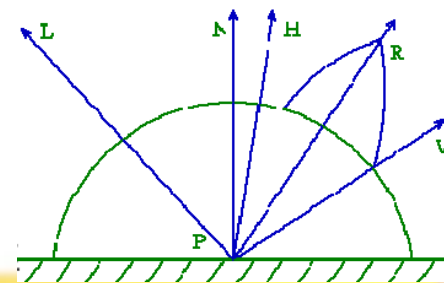


- Diffuse reflection(漫反射光效果)
 - The propagation of diffuse reflection rays is isotropic.
(漫反射光的传播是各向同性的)
 - 漫反射光强近似服从Lambert定律, the intensity of diffuse reflection is defined as(漫反射光的强度为)

$$I_d = I_p K_d * (L \cdot N)$$

- K_d is the diffuse reflection constant, I_p 点光源发出的入射光强度
- K_d has three components k_{dr}, k_{dg}, k_{db} represent R,G,B diffuse reflection constant respectively.

Phong Lighting Model



- Specular reflection(镜面反射光效果)
 - For glossy(光滑的) surfaces, the reflected rays always exist in a very narrow solid angle area, determined by the direction of reflection law(依据反射定律，反射光线往往集中在一个小的立体角内，这些反射光我们称之为镜面反射光，其特点是在光滑表面会产生高光区域。注意，镜面反射产生的颜色只是光源的颜色，在白光的照射下，高光点就是白色)

- The intensity of specular reflection is defined as:

$$I_s = I_p K_s * (R \cdot V)^n$$

- K_s is the specular reflection constant ; and n is the shininess constant(反射指数), which decides how shiny the surface is.(n 越大，表面物体表面越光滑，则高光区域越集中)



Phong Lighting Model

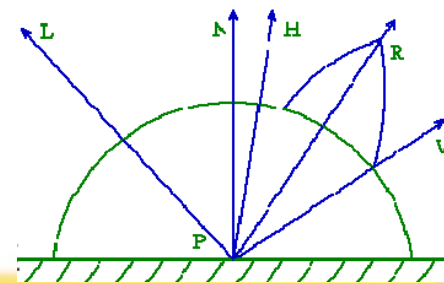
- Ambient reflection(环境光效果)
- 环境光：邻近各物体所产生的光的多次反射最终达到平衡时的一种光。可近似认为同一环境下的环境光，其光强分布是均匀的。
是对间接光照的简单模拟。

– The intensity of ambient reflection is defined as:

$$I_e = I_a K_a$$

- K_a is the ambient reflection constant(物体对环境光的反射系数), I_a 为环境光的光强。

Phong Lighting Model



- The reflection intensity is the sum of diffuse reflection, specular reflection and ambient reflection(由物体表面上一点 P 反射到视点的光强 I 为环境光的反射光强+理想漫反射光强+镜面反射光强)

假定只有一个光源

$$I = I_a K_a + I_p K_s * (R \cdot V)^n + I_p K_d * (L \cdot N)$$

环境光的反射光强

镜面反射光强

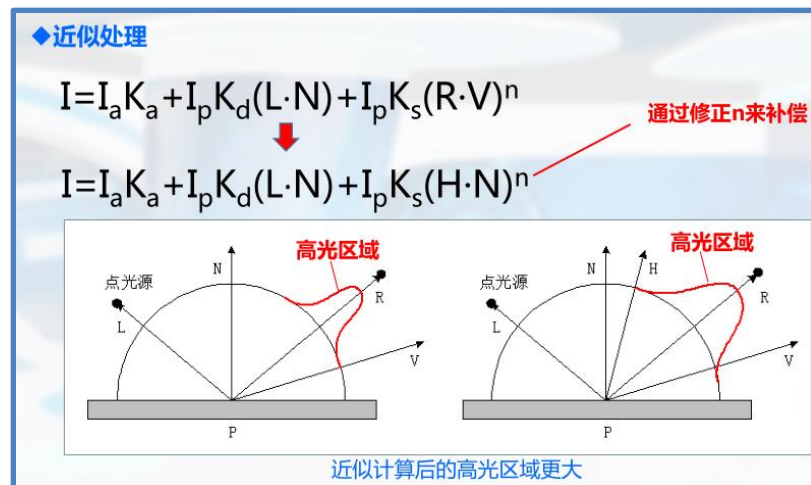
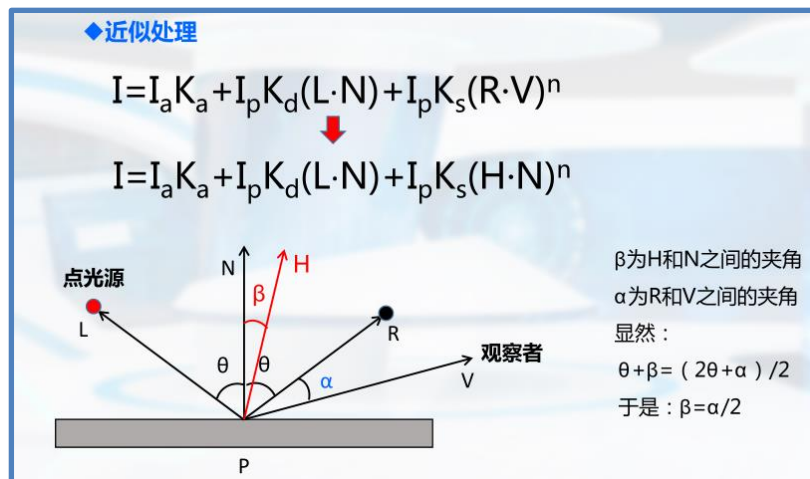
理想漫反射光强

- 这就是经典的Phong模型。 I_a (环境光的光强)、 I_p (入射光强) 都是常数, K 也是已知的, L 是光源的方向也是已知的, N 是物体表面的法向可以算出来的, V 是视线的方向, R 也可以算出来。



Phong Lighting Model的实现

- 用Phone模型进行真实感图形计算时，对物体表面上的每个点P，均须计算光线的反射方向R。再由R计算 $(R \cdot V)$ ，其中V是视点方向。为减少计算量，可作如下假设：
- 光源在无穷远处，即光线方向L为常数。
 - 视点在无穷远处，即视线方向V为常数。
 - 用 $(H \cdot N)$ 近似 $(R \cdot V)$ 。这里对所有点总共只需计算一次H值，为L和V的平分向量；另外H的计算比R的计算简单。



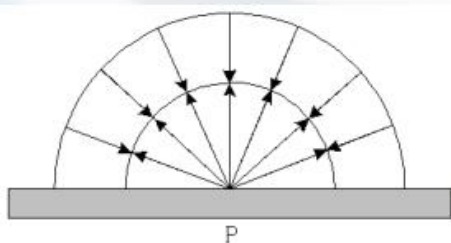


Phong Lighting Model的实现

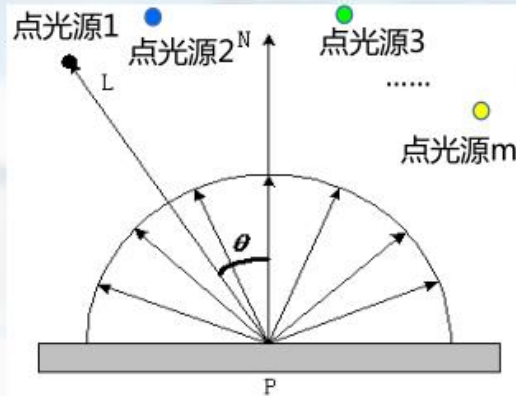
◆多个光源 (m个点光源)

光强计算假定只有一个点光源，若在场景中有m个点光源，则可以在任一P点上叠加各个光源所产生的光照效果：

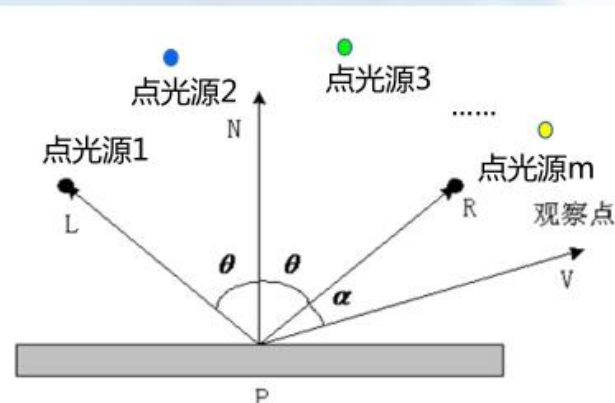
$$I = I_a K_a + \sum_{i=1}^m I_{p,i} K_d (L_i \cdot N) + \sum_{i=1}^m I_{p,i} K_s (H_i \cdot N)^n$$



环境光
Ambient



漫反射
Diffuse



镜面反射
Specular



Phong Lighting Model的实现

□ Phong光照模型：彩色的光照明场景

- 选择合适的颜色模型, 假定选择RGB颜色模型
- 为颜色分量分别建立光照模型

结合RGB颜色模型， Phong光照明模型最终有如下的形式：

$$\begin{cases} I_r = I_{ar} K_{ar} + I_{pr} K_{dr} (L \cdot N) + I_{pr} K_{sr} (H \cdot N)^n \\ I_g = I_{ag} K_{ag} + I_{pg} K_{dg} (L \cdot N) + I_{pg} K_{sg} (H \cdot N)^n \\ I_b = I_{ab} K_{ab} + I_{pb} K_{db} (L \cdot N) + I_{pb} K_{sb} (H \cdot N)^n \end{cases}$$

- ## □ Phong光照模型的缺点：
- 用phone模型显示出的物体像塑料，没有质感；环境光是常量，没有考虑物体之间相互的反射光；镜面反射的颜色是光源的颜色，与物体的材料无关；镜面反射的计算在入射角很大时会产生失真等。

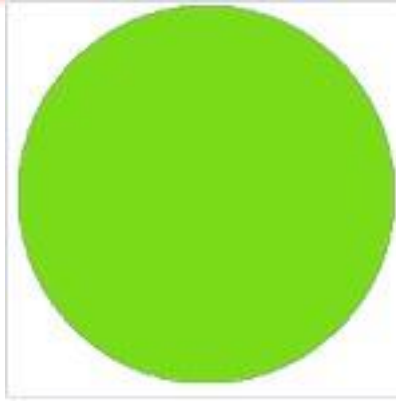


Example 1



Diffuse
(漫反射)

+



Ambient
(环境光)

+



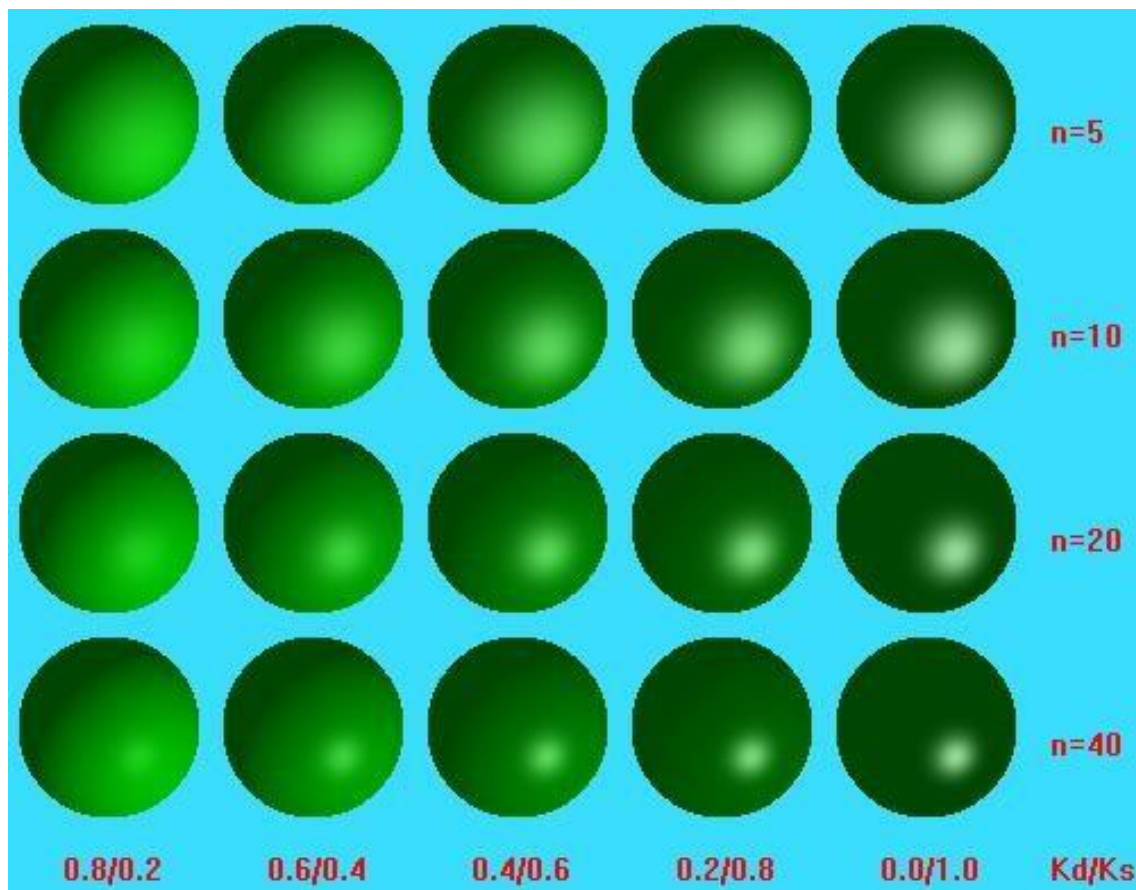
Specular(境
面反射光)

=





Example 2



- 反射指数 n 控制高光的“集中程度”（即材质的光滑度）： n 越小，高光范围越大，材质显得越粗糙； n 越大，材质显得越光滑。
- 高光几乎与漫反射混合，改变 K_d/K_s 主要影响整体亮度。
- K_d 与 K_s 的比例控制“漫反射与镜面反射的强度比重”：漫反射决定物体固有色在光照下的明暗分布，镜面反射决定高光的亮度和颜色（通常为光源色）。
- K_d 减小使漫反射区变暗，高光相对更突出。 K_d 很小时（如 0.2/0.8）物体大部分区域暗，仅高光亮。高光本身亮度由 K_s 主导。



Wu Jianhua's Demo

Demo

<http://cg.cs.tsinghua.edu.cn/course/>



Shading(明暗处理)

□ 为什么要进行明暗处理？

- Mach Band (马赫带效应)

三维物体通常用多边形来近似模拟。

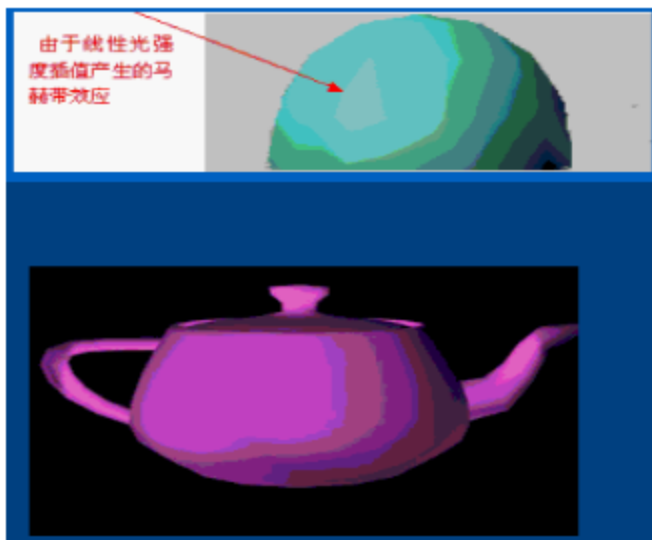
由于每一个多边形的法向一致，因而**多边形内部的像素的颜色都是相同的**，而且在不同法向的多边形邻接处，光强突变，使具有不同光强的两个相邻区域之间的**光强不连续性**(马赫带效应)



Shading(明暗处理)

马赫带效应举例

相邻区域之间的光强不连续性，亮暗强度条纹



为了保证多边形之间的光滑过渡，使连续的多边形呈现匀称的光强分布，多边形细分效率不高

解决办法？ ➡ 增量式光照模型

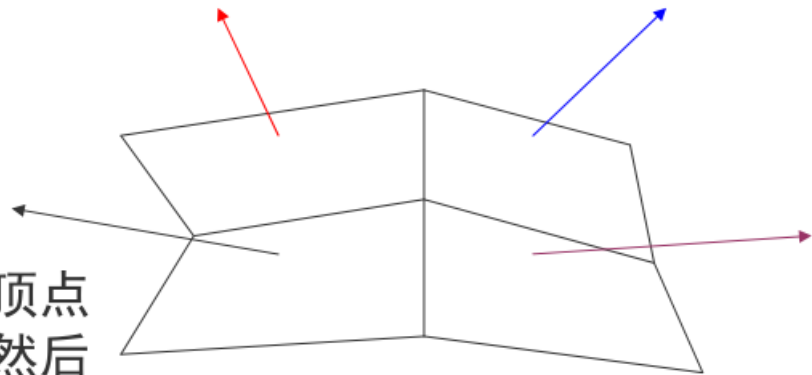


Shading(明暗处理)

□ 如何进行明暗处理？

- **Solution:** Lighting or other parameters are calculated in each vertex of the polygon, then in each point inside the polygon, the intensity of lighting is computed through bicentric interpolation between the vertices.

基本思想：每一个多边形的顶点处计算出光照强度或参数，然后在各个多边形内部进行均匀插值





Shading

- Two main algorithms
 - Gouraud shading: interpolation of intensity.
 - Phong shading: interpolation of normal.

双线性光强插值

双线性法向插值



Gouraud shading

- Proposed by Gouraud in 1971
 - called Gouraud interpolation(双线性光强插值)
 - 又称为**亮度插值**明暗处理，它通过对多边形顶点**颜色**
进行线性插值来绘制其内部各点。
- Computation
 - Four **steps**

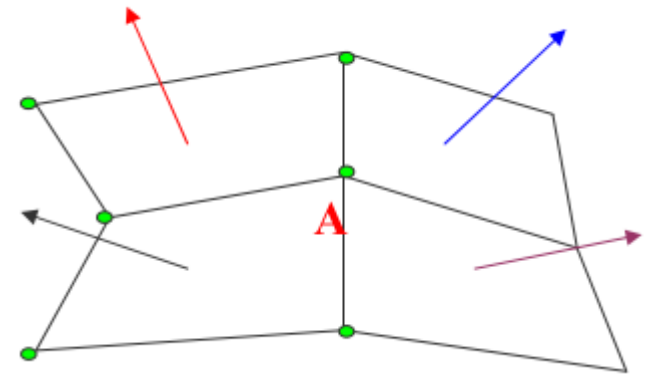


Gouraud shading

Step1: 计算多边形顶点的平均法向。

与某个顶点相邻的所有多边形的法向平均值近似作为该顶点的近似法向量，顶点A相邻的多边形有k个，它的法向量计算为：

$$N_a = \frac{1}{k} (N_1 + N_2 + \dots + N_k)$$



Step2: 用简单光照模型(如Phong光照模型)计算顶点的光强。

Phong光照模型出现前，采用如下光照模型计算：

$$I = I_a K_a + I_p K_d (L \cdot N_a) / (r + k)$$

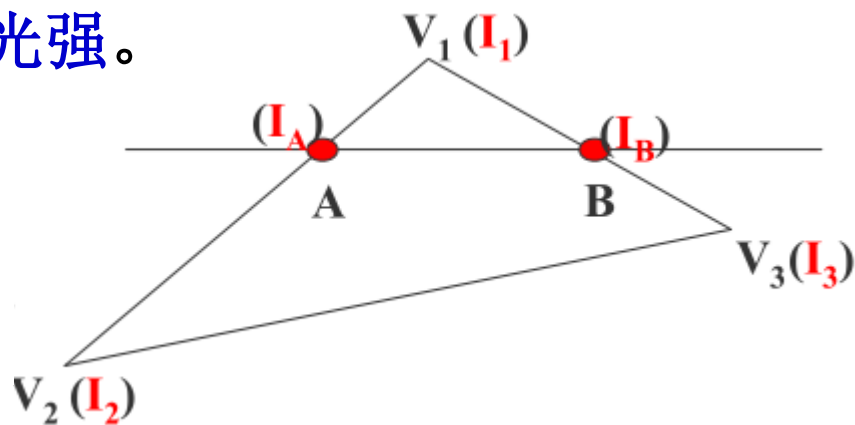


Gouraud shading

Step3: 插值计算离散边上各点的光强。

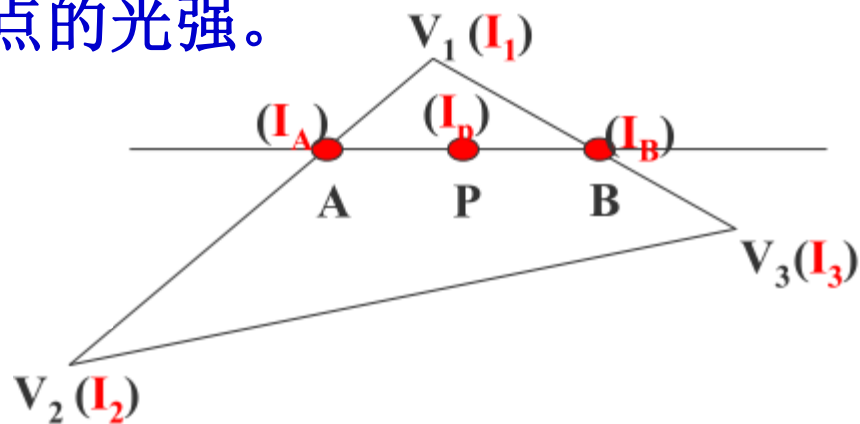
$$I_A = uI_1 + (1 - u)I_2 \quad u = \frac{AV_2}{V_1V_2}$$

$$I_B = vI_1 + (1 - v)I_3 \quad v = \frac{BV_3}{V_1V_3}$$



Step4: 插值计算多边形内域中各点的光强。

$$I_p = tI_A + (1 - t)I_B \quad t = \frac{PB}{AB}$$



- 增量计算



Phong shading

Gouraud明暗处理的不足：

- Gouraud明暗处理解决了相邻多边形之间颜色突变问题，使得颜色过渡均匀，但是由于采用光强插值，它的镜面反射效果不太理想，而且相邻多边形边界处的马赫带效应并不能完全消除。
- 解决办法？ Phong明暗处理，
- 双线性法向插值
- 以时间为代价，引入镜面反射，解决高光问题



Phong shading

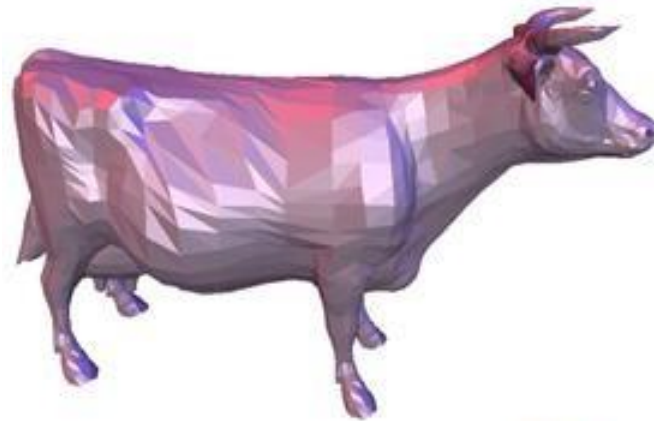
- Developed by Bui Tuong Phong in his 1973 PhD dissertation
 - Note: **it is different from Phong lighting model**
- Computation
 - Different from Gouraud shading, it does not interpolate color values, it interpolates surface normal, and use the interpolated normal to calculate the color value. (Phong明暗处理不是对颜色插值，而是对**点的法向量**进行插值，得到**连续的法向量场**，再利用该法向量场逐点使用光照模型进行色彩的计算)



Phong shading

Comparison

only by lighting model
(直接用光照明模型进行绘制)



by Phong shading





Phong shading

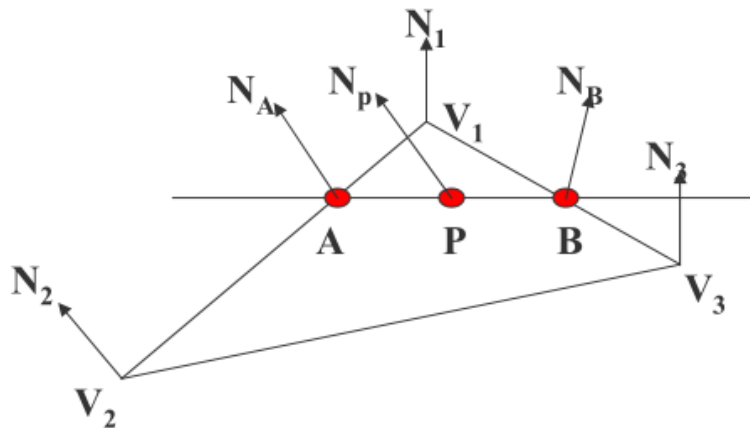
- **Phong明暗处理方法**，又称为**法向量插值明暗处理**，它对多边形顶点的法向量进行插值以产生中间各点的法向量，其**步骤**为：
- 计算每个多边形顶点处的平均单位法向量；
 - 用双线性插值方法求得**多边形内部各点的法向量**。
 - 最后按光照模型确定多边形内部各点的光强。



Phong shading

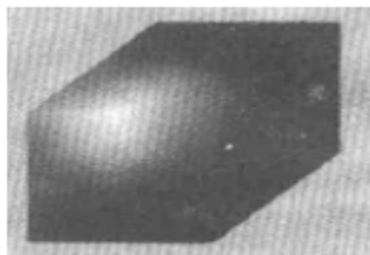
Phong明暗处理是先算角点的法向量，再算内部点的法向量，最后再用新的光照模型算内部点的颜色值。

点A的法向量 N_A 为 N_1 与 N_2 的线性插值，点B的法向量 N_B 为 N_1 与 N_3 的线性插值，点P的法向量 N_P 为 N_A 与 N_B 的线性插值。

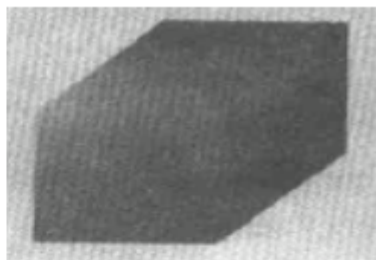




Phong shading



Phong方法



Gouraud方法

Phong方法	Gouraud方法
1、产生的效果高光明显； 2、高光多位于多边形内部； 3、明暗变化缺乏层次感。	1、效果并不明显； 2、多边形内部无高光； 3、光强度变化均匀，与实际效果更接近。

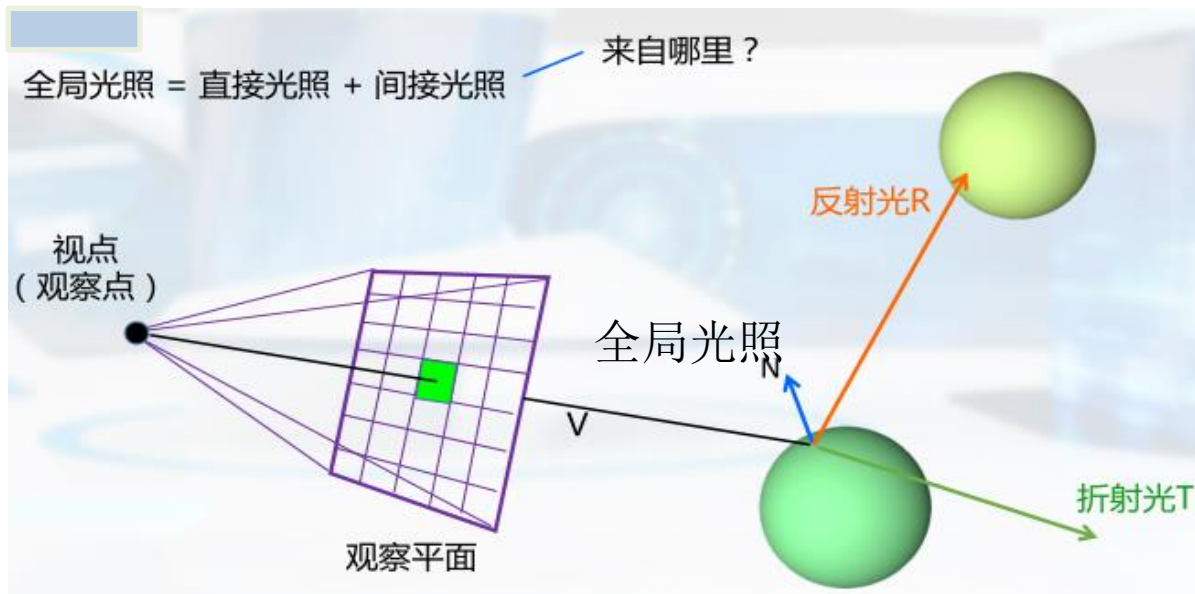


整体光照模型

- 为什么需要整体光照模型？
 - 简单和局部光照模型不能很好地模拟光的折射、反射和阴影等，也不能用来表示物体间的相互光照影响。
- 一个完整的光照明模型应该包括由光源和环境引起的漫反射分量、镜面反射分量、规则透射分量以及漫透射分量等。
 - 仅考虑由光源引起的漫反射分量和镜面反射分量，而环境反射分量则简单地用一常数来代替，这类光照模型称为局部光照模型。
 - 能同时模拟光源和环境照明效果的光照模型称为整体光照模型。



全局光照



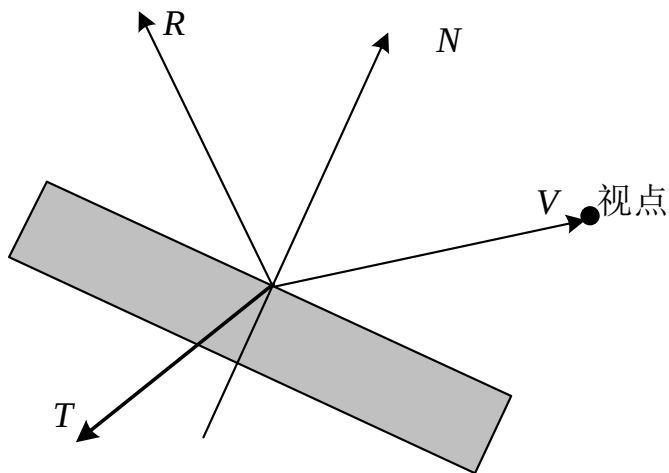
- 一个来自其他物体反射后照射过来的光照；
- 另一个呢？若这个物体具有一定的透明度，则可以透过这个物体看见折射光呢？
- 于是**Phone**模型就不适用，一种新的模型：**Whitted**光透射模型被提出。



Whitted光透射模型

- 简单和局部光照模型只考虑了由光源引起的漫反射和镜面反射，没有处理光的透射现象。光透射模型把透射现象的模拟引入真实感图形中。
- Whitted在简单光照模型中增加了环境镜面反射光和环境规则透射光，以模拟周围环境的光投射在景物表面上产生的理想镜面反射和规则透射现象。

Whitted光透射模型 = Phong模型 + 透射光强 + 反射光强(对其余物体反射光的反射)



首先假定透过这个物体能看见折射光
即能看见透射光的物体

可以看见透射出的折射光

透射光强 = $I_t K_t$

其中：

I_t 是折射方向光强

K_t 是透射系数

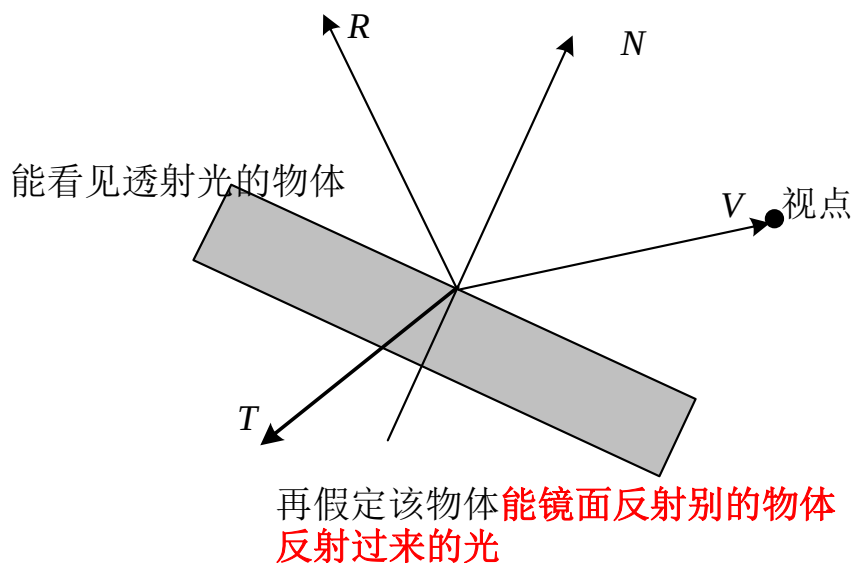
Whitted光透射模型

$$I = I_a K_a + I_p K_d (L \cdot N) + I_p K_s (R \cdot V)^n + I_t K_t$$



Whitted光透射模型

Whitted光透射模型 = Phong模型 + 透射光强 + 反射光强(对其余物体反射光的反射)



能反射别的物体反射过来的光

反射光强 = $I_r K_r$

其中：

I_r 是入射光强度（来自别的物体的反射）

K_r 是反射系数

Whitted光透射模型

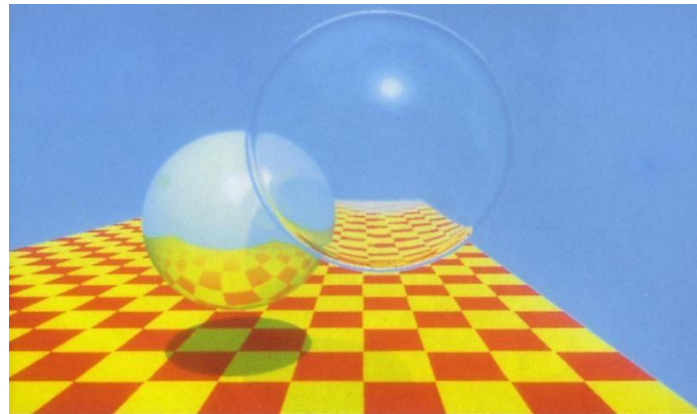
$$I = I_a K_a + I_p K_d (L \cdot N) + I_p K_s (R \cdot V)^n + I_t K_t + I_r K_r$$

- 基于简单光照明模型的光透射模型，虽然可以模拟光的折射，但是这种折射的计算范围很小，也不能很好的模拟多个透明体之间复杂的光照明现象。为解决该问题，需要一个更精确的光照明模型，即整体光照明模型，其中光线跟踪是一种主要的整体光照明模型。



History of Ray Tracing (光线跟踪)

- In 1980, Whitted proposed a ray tracing model, include light reflection and refraction(折射)effects. A Milestone of Computer Graphics. (计算机图形学历史上的里程碑)



- Turner Whitted , An improved illumination model for shaded display, Communications of the ACM, v.23 n.6, p.343-349, June 1980.



History of Ray Tracing (光线跟踪)

- After receiving his PhD from NCSU in 1978, Turner Whitted left for Bell Labs(贝尔实验室) and proceeded to shake the CGI world with an algorithm that could ray-trace a scene in a reasonable amount of time.(提出了震惊世界的光线跟踪算法)
- He only has 19 papers, and Ray tracing is his first paper.
- He was elected as member of National Academy of Engineering in 2003.(当选美国工程院院士)



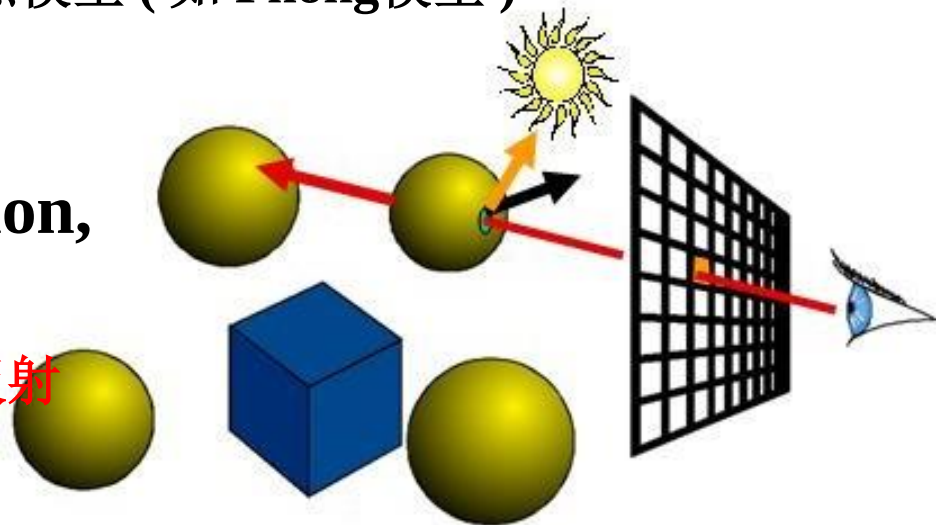
Introduction of ray tracing

- **Ray tracing**
 - **Because of its effectiveness, ray tracing is a widely used and very powerful rendering (drawing) technique**(它作为一个有效的真实感图形绘制算法被广泛地使用)
 - **Why we see objects?**
 - **Light can be interpreted as a collection of rays that begin at the light sources and bounce around the objects in the scenes.**(光可以理解为一组由光源发出经物体反复弹射而最终进入视点的光线)
 - **We see objects because rays finally come into our eyes.**
(最终将光线射入人们的眼中，使我们看到物体)

Basic Idea of ray tracing(如何让计算机实现图形绘制呢:是反过来做的, 光线跟踪方向与光传播方向是相反的)

- For each pixel, what can we “see”? (对于每个像素 P 计算其色彩值)
- a ray casting from the eye through the center of the pixel and out into the scene, its path is traced to see which object the ray hits first (光线从眼睛穿过像素的中心投射到场景中, 计算由视点连接像素 P 中心的光线 延长后所碰到的第一个物体的交点)
- calculate the shading value of the point by the Phong model (使用局部光照模型 (如 Phong 模型) 计算交点处的颜色值)
- continue to trace the ray in the scene to achieve reflection, refraction..

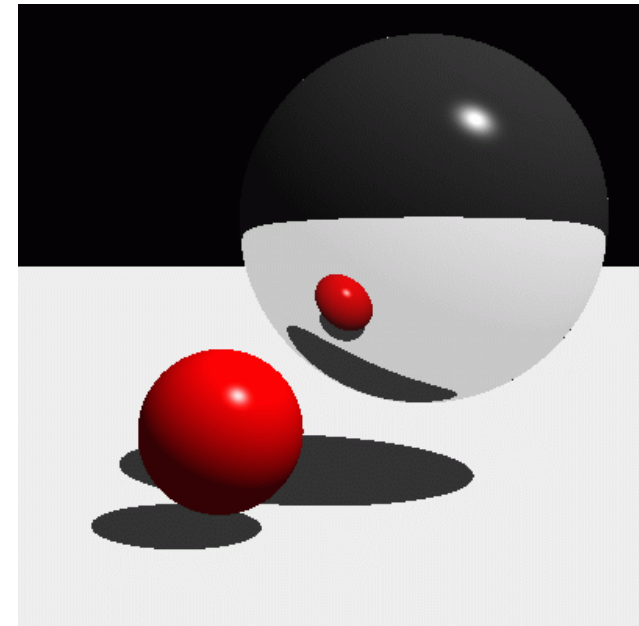
(沿交点处的反射和折射, 继续向反射或折射方向跟踪另一条光线)





Features of ray tracing

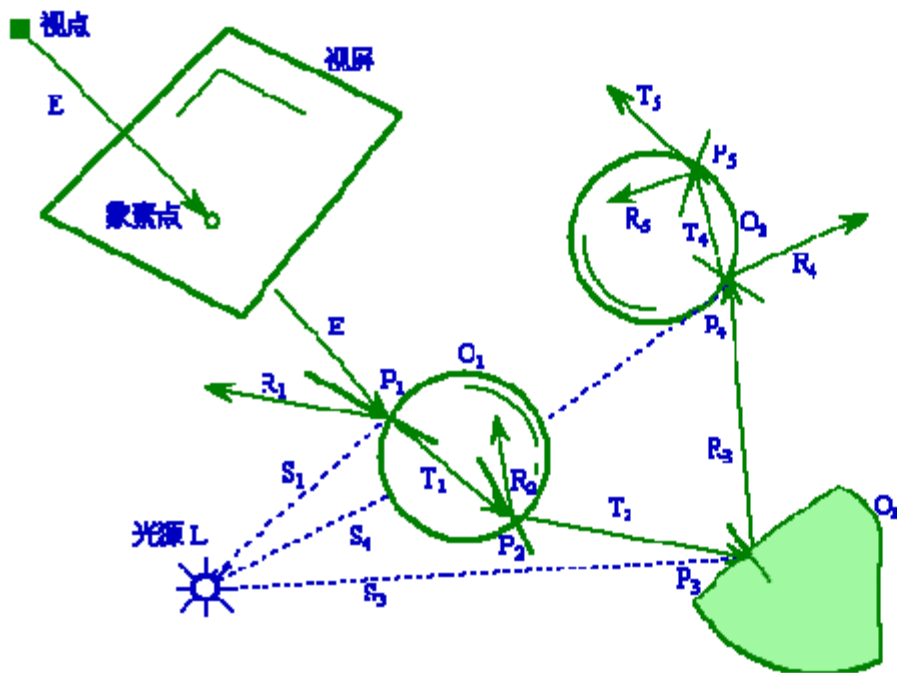
- **Features(光线跟踪的特征)**
 - **Easy to incorporate interesting visual effects, such as shadowing, reflection and refraction, since the path of a ray is traced through the scene(通过光线跟踪, 可以很容易地表现出例如阴影、反射、折射等引人入胜的视觉效果)**
 - **Besides geometric primitives (such as spheres, cones, cubes), easy to work with a richer class of objects, including polygonal meshes, compound objects.(除了基本的几何形体, (例如球体、椎体、立方体等), 光线跟踪容易适用于更复杂的物体表示方法 (例如多边形网格表示或者复合形体等))**





Basic procedure of ray tracing

光线跟踪基本过程

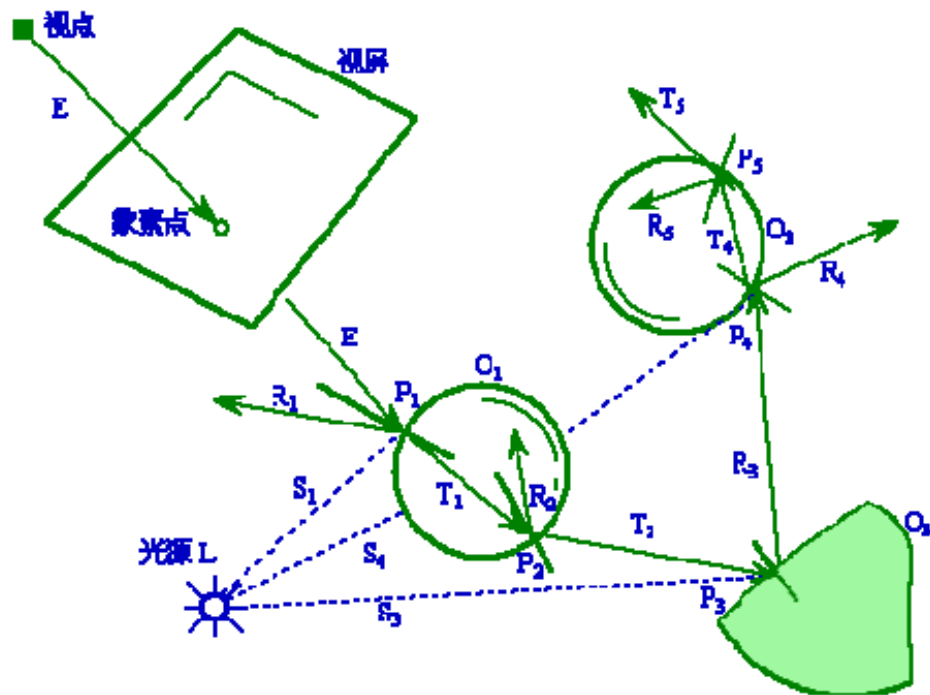


在这个场景中，
有一个点光源 L ，两
个透明体 O_1 与 O_2 ，
一个不透明体 O_3 。



Basic procedure of ray tracing

光线跟踪基本过程

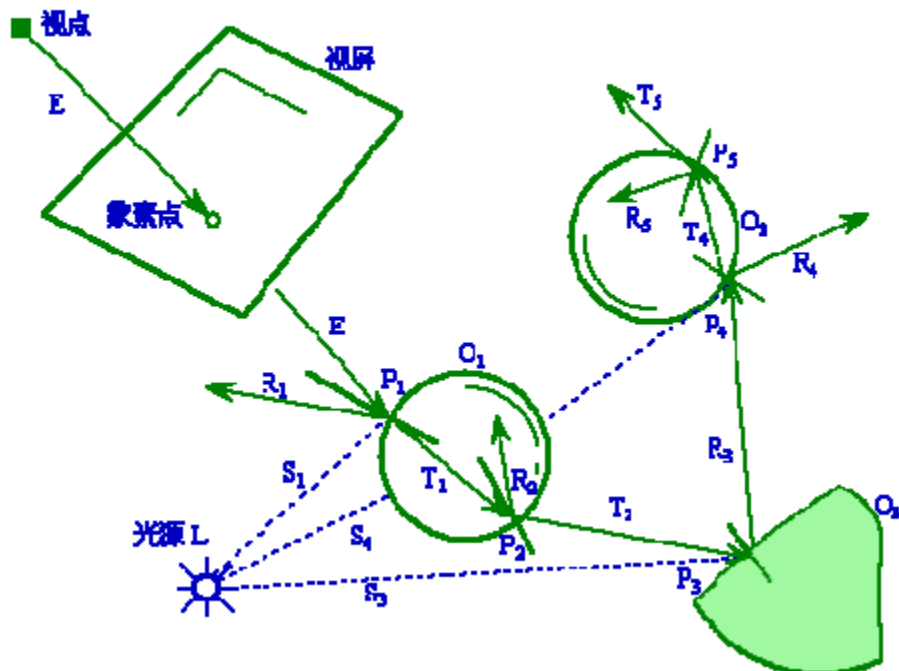


在反射光线 R_1 方向上，没有再与其他物体相交，那么就设该方向的光强为0，并结束这条光线方向的跟踪。然后对折射光线 T_1 方向进行跟踪，计算该光线的光强贡献。



Basic procedure of ray tracing

光线跟踪基本过程



折射光线 T_1 在物体 O_1 内部传播，与 O_1 相交于点 p_2 ，由于该点在物体内部，假设它的局部光强为0。该点处同时产生了反射光线 R_2 和折射光线 T_2 ，在反射光线 R_2 方向，可以继续递归跟踪下去计算它的光强。而对折射光线 T_2 则继续进行跟踪。



Recursive Ray Tracing

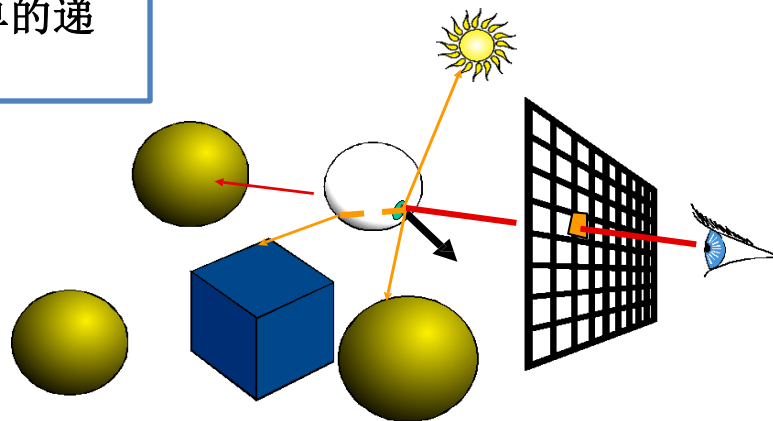
过程及原理总结：从视点出发，向每个像素发出射线，跟场景中物体求交点。若光线与场景中景物无交，则光线将射出画面，跟踪结束。否则，光线与景物有交。此时，光线在离视点最近的景物表面交点处的走向有以下三种可能：

- 当前所在的景物表面为理想镜面，光线沿其镜面反射方向继续跟踪；
- 当前交点所在的景物表面为规则透射面，光线沿其规则透射方向继续跟踪；
- 跟踪结束。

实现：在第一个交点处计算它的局部光强，然后看它的折射方向和反射方向，把它们在折射方向和反射方向上由其他物体引起的间接光强给它递归回来，乘上系数累加回来。这样一个复杂的图形绘制问题变成这么简单的递归来实现。

光线跟踪是从 ** 摄像机（眼睛） ** 出发，向场景里发射一条光线，追踪它和物体的交互：

- 碰到物体：计算物体本身的颜色、高光
- 碰到镜面：生成一条**反射光线**，继续追踪
- 碰到透明 / 半透明物体：生成一条**折射光线**，继续追踪
- 被物体挡住、碰不到光源：就会形成**阴影**

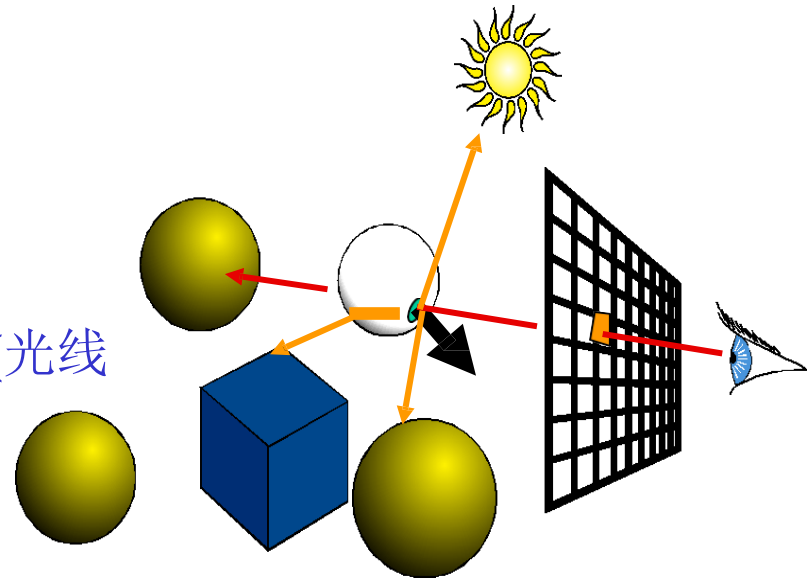


光线 V 与物体表面交于交点 P 时，光在交点 P 对光线 V 方向的贡献分成三部分：交点处的局部光强，反射方向上由其他物体引起的间接光强（通过对反射光线的递归跟踪得到），折射方向上由其他物体引起的间接光强（通过对折射光线的递归跟踪得到）。这三部分光强相加，就是这条光线 V 在 P 点处的总光强。



Recursive Ray Tracing

- **Does it ever end?** **Stopping criteria:**
- Recursion depth(递归深度)
 - Stop after a number of bounces(光线弹射一定次数)
- Ray contribution(光线的贡献)
 - Stop if reflected / refracted contribution becomes too small(在光线的贡献衰减到足够小时停止)
- 该光线未碰到任何物体。
- 该光线碰到了背景。





Recursive Ray Tracing

```
IntersectColor( vBeginPoint, vDirection)
{ // vBeginPoint光线起点, vDirection: 光线方向
  Determine IntersectPoint;
  //计算第一个交点
  Color = ambient color;
  for each light
    Color += local shading term; //交点处用局部光照模型计算出的光强
  if(surface is reflective)
    color += reflect Coefficient * //追踪反射光线
      IntersectColor(IntersecPoint, Reflect Ray);
  else if ( surface is refractive)
    color += refract Coefficient * //追踪折射光线
      IntersectColor(IntersecPoint, Refract Ray);
  return color;
} //第一次调用该函数时, 起点是视点, 方向为视点到该像素点的射线方向
//如果交点处表面是镜面或折射面, 则继续向反射或折射方向跟踪另一条光线
```

Recursive Ray Tracing



RayTracing(start, direction, weight, color)

```
{  
  if ( weight < MinWeight )  
    color = black;  
  else  
  {  
    计算光线与所有物体的交点中离 start 最近的点;  
    if ( 没有交点 )  
      color = black;  
    else  
    {  
       $I_{local}$  = 在交点处用局部光照模型计算出的光强;  
      计算反射方向 R;  
      RayTracing(最近的交点, R, weight*  $w_r$ ,  $I_r$ );  
      计算折射方向 T;  
      RayTracing(最近的交点, T, weight*  $w_t$ ,  $I_t$ );  
  
      color =  $I_{local} + K_s I_r + K_t I_t$ ;  
    }  
  }  
}
```

start: 光线的起点;

direction: 光线的方向

weight: 光线的衰减权重

color: 算法最后返回光线方向上的颜色值

//递归得到的反射方向上由其他物体引起的间接光强

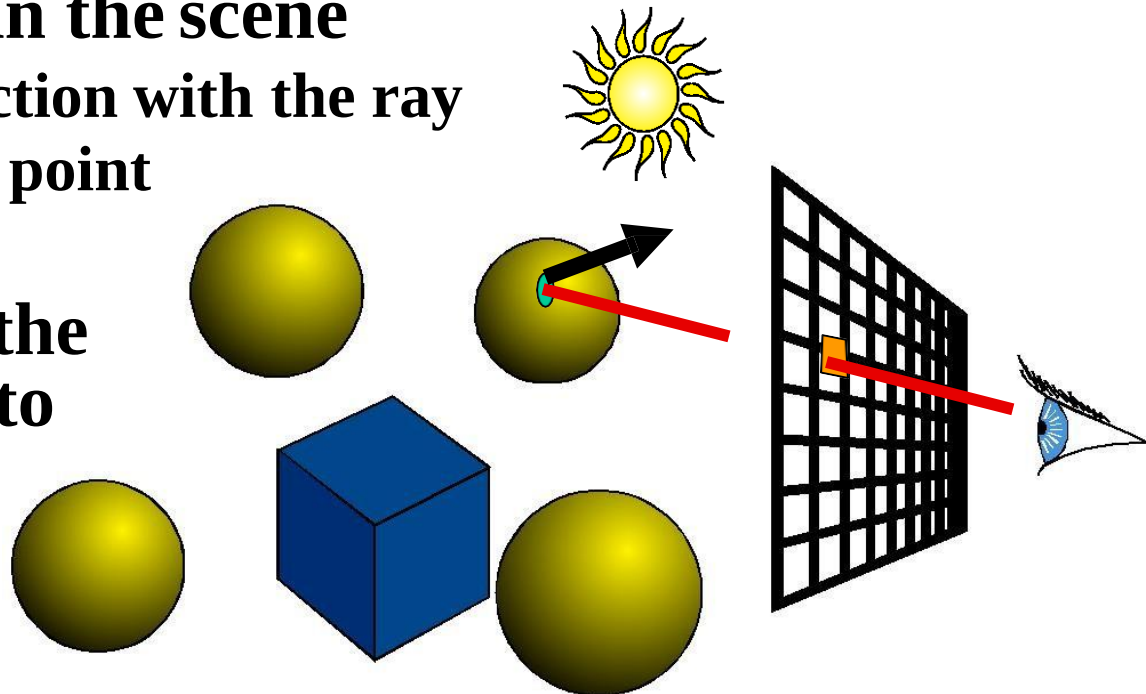
//递归得到的折射方向上由其他物体引起的间接光强



Ray Casting

光线投射 (Ray Casting): 光线追踪算法中的第一步

- **For each pixel**
 - **Cast a ray from the eye, through the center of pixel, to the scene** (由视点向该像素中心投射一条光线, 射向场景当中)
 - **For each object in the scene**
 - Find the intersection with the ray
 - Store the closest point
 - **Calculate local shading term of the point according to light, material and normal**
(使用局部光照模型计算像素颜色值)





Ray Casting

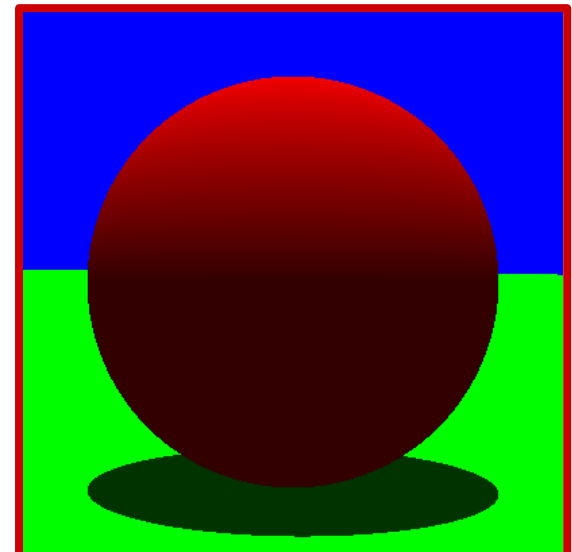
- **Shading results depend on surface normal, light direction, light intensity, view direction, material property and so on**(明暗效果仅仅由第一次相交的物体表面法向方向、材质、视点和光照方向、以及光照强度等因素共同决定)
- **Do not account for secondary ray, so do not have shadow, reflection and refraction effects**(光线投射并不考虑第二层以及更深层次的光线，因此不具有阴影、反射、折射等效果)



Add Shadows(添加阴影)

- For each pixel
 - Cast a ray and find the intersection point
 - color = ambient
 - for each light
 - if intersection point is not in shadow area of the light (evaluated by shadow rays)
color += shading from this light
 - return color

从交点发出射向光源的**阴影测试线**，若阴影测试线被物体遮挡，它就在阴影区域里。





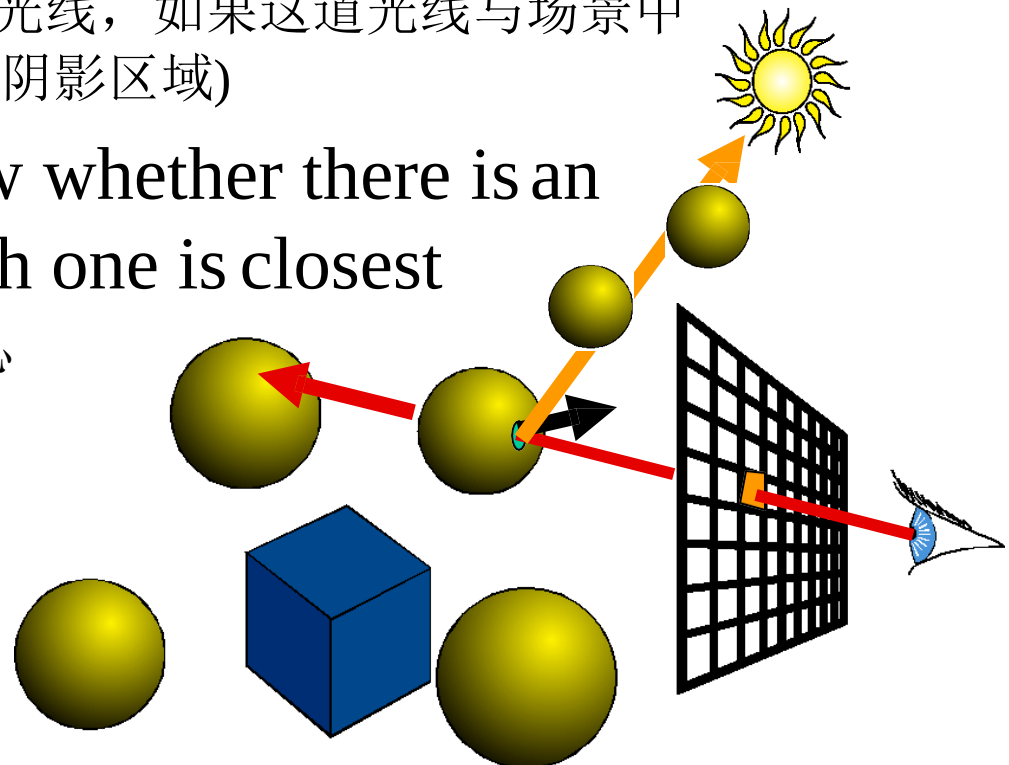
Add Shadows

- As illustrated in the figure below, casting a shadow ray from intersection point to the light, if there is an intersection, this point is in shadow

(由交点向光源方向投射一道光线，如果这道光线与场景中物体有交点，则该交点属于阴影区域)

- We only want to know whether there is an intersection, *not* which one is closest

(计算阴影区域时，我们只关心是否与物体相交，而不关注哪个是最近交点)



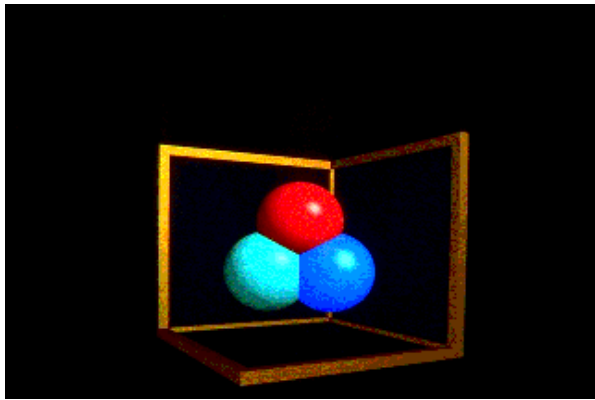


Add Ray reflection and refraction (添加反射和折射效果)

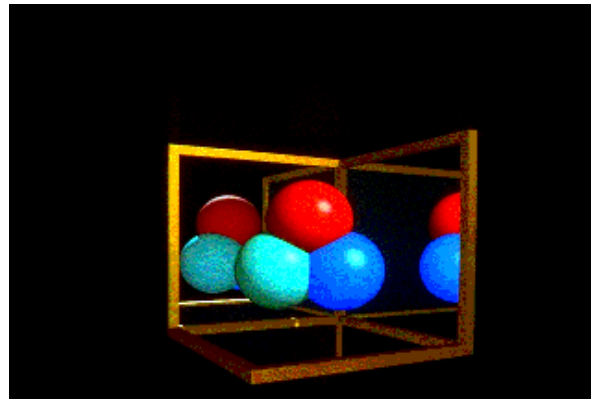
- Ray tracing gives the ability to have objects with mirror reflections or objects with refractions.(光线跟踪具有模拟物体表面镜面反射效果以及折射效果的能力)
 - The first step is to determine where a ray intersects the object.(计算光线与场景中物体的最近的交点)
 - The next step is to determine the direction the ray will travel when it reflects off the surface or refracts through the object.(计算光线在交点处被物体反射和折射所产生的新的光线的方向)
 - a new ray direction is calculated based on the incoming ray and the surface normal.(新的方向由入射光方向、物体表面法向及介质共同决定)
 - 对新产生的光线分别继续进行跟踪。



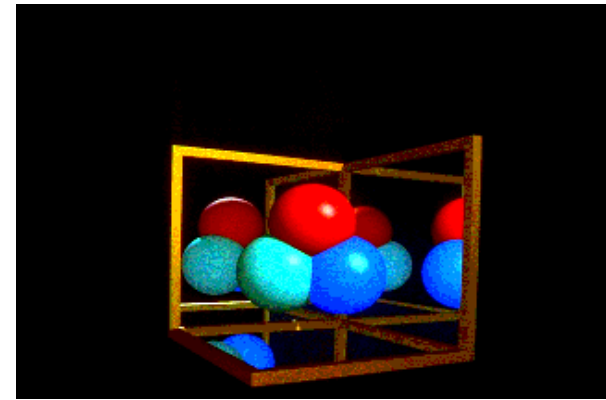
Recursion ray tracing results



0 recursion



1 recursion



2 recursions

- 0 次递归：光线只做一次计算，不考虑任何反射效果。它完全不追踪任何反射光线，所以镜面里不会出现任何镜像，也看不到球体在镜子里的倒影。
- 1 次递归：光线在碰到物体后，可以再反射一次（追踪一次反射光线）。镜面会反射出球体的倒影，这就是第一次反射的结果。镜子里出现了球体的镜像，但这些镜像的“再反射”是不存在的，镜像里的球体，不会再在镜子里反射出自己的倒影。
- 2 次递归：允许光线反射两次。第一次反射：主光线打到球体，镜面里出现第一次镜像。第二次反射：第一次镜像的光线，又被另一个镜面反射，形成“镜像的镜像”。



第8讲课后作业:

教材习题4（第169页）：2、4、5、9、10（只需要回答递归终止条件有哪些即可）



Thanks!